Algorithm 1: $(\texttt{RolloutStorage}_g)_{g\in\mathcal{G}} \leftarrow \texttt{PartitionedQueueRollout}(\theta_k, \mathsf{m}, G_k)$

Parameters: finite PBG $(\mathcal{S}, \mathcal{T}, \mathcal{P}, o, (\mathcal{E}_\sigma)_\sigma, \mathcal{X}, \mu, \chi)$; read-only π^{θ_k}; $\mathsf{m} \in \{\mathsf{normal}, \mathsf{seqRB}, \mathsf{branchRB}\}$; GPU set $\mathcal{G}$; in sequential modes the common per-GPU root width G_k chosen before current draws by `ChooseSequentialRootWidth` in `sequential_memory_control.tex`

Parameters: finite opaque partition interface $\mathfrak{M} = (\mathcal{M}, \kappa, \preceq, (\texttt{PartitionSpec}_m)_{m\in\mathcal{M}})$: $\kappa(\sigma)$ routes player nodes, $\preceq$ is an optional preorder, and $\texttt{PartitionSpec}_m$ supplies natural input/model-row domains $(\mathcal{X}_m, \mathcal{Y}_m, \varphi_m)$, a model boundary, fixed worst-case bytes/work per row, and safe batch limits

Parameters: for branchRB only, public frozen $\beta : \mathcal{S} \setminus \mathcal{T}^{\mathsf{pre-c}} \to [1, \infty)$

Definitions: $\mathcal{T}_{\mathsf{normal}} = \mathcal{T}$ and $\mathcal{T}_{\mathsf{seqRB}} = \mathcal{T}_{\mathsf{branchRB}} = \mathcal{T}^{\mathsf{pre-c}}$; $n(s) = 1$ in sequential modes and

$$n(s) = \lfloor \beta(\ell(s)) \rfloor + \mathrm{Bernoulli}\big(\beta(\ell(s)) - \lfloor \beta(\ell(s)) \rfloor; K(k, \iota(s), \mathsf{round})\big)$$

in branchRB, including at the distinguished root $\varnothing$

Definitions: $\iota(s)$ is a placement-independent logical address. Every random draw uses a disjoint counter key $K(k, \iota(s), u, \mathsf{purpose})$; hence queue order, physical pack size, and GPU speed cannot change the sampled object

Definitions: In sequential modes $\mathcal{I}_g = \{(g, i) : 0 \le i < G_k\}$. `SafeAdmit` controls only how many of these already-fixed roots are live at once; it may neither change G_k nor discard, retry, or replace a trajectory

Definitions: $\texttt{Pack}_m(Q)$ removes a stable insertion-order prefix of Q. $\texttt{PhysicalPlan}_m(B)$ returns stable maximum-feasible subspans of B; it neither sorts nor reroutes rows. $\texttt{Full}_m(Q)$ and all memory admission tests use the partition's declared worst-case physical contract, not an undeclared game label

Definitions: Any failed hard transaction sets a local failure flag and exits current work without retrying a draw. Every rank reaches the single common-order failure agreement after local rollout; no rank enters targets or mass collectives with a partial store

Definitions: $\mathrm{Min}_{\preceq} C = \{m \in C : \nexists m' \in C,\ m' \prec m\}$. Among full queues choose an element of $\mathrm{Min}_{\preceq} C$ by decreasing $(\texttt{load}, \texttt{age})$; when forced to drain partial work, first restrict to the minimal nonempty preorder layer and then maximize $(\texttt{fill}, \texttt{age})$

Output: one GPU-local immutable $\texttt{RolloutStorage}_g$ containing only a mode-specific compact topology, stable partition tables, and leaf rows

foreach $g \in \mathcal{G}$ ***in parallel*** **do**

  Allocate a chunked device $\texttt{OccurrenceArena}_g$, append-only partition tables, bounded reusable staging, $f_g \leftarrow 0$, and $\mathcal{Q}_{g,m} = \varnothing$ for all m;

  if $\mathsf{m} = \mathsf{branchRB}$ **then**

    Create exactly one occurrence $s_0 = \varnothing$ with $\ell(s_0) = \varnothing$, reach $r_{s_0,q} = 1$, and $\log \omega(s_0) = 0$; set $\iota(s_0) = (g, \varnothing)$; append $(P, L) \leftarrow \texttt{ChanceClose}_g(\{s_0\}, \mathsf{m})$ and enqueue each $s \in P$ into $\mathcal{Q}_{g,\kappa(\ell(s))}$; $U \leftarrow \varnothing$;

  else

    $U \leftarrow \mathcal{I}_g$ `/* unstarted root trajectories */`

  end

  while $U \neq \varnothing$ *or* $\bigsqcup_m \mathcal{Q}_{g,m} \neq \varnothing$ **do**

    $F \leftarrow \{m : \texttt{Full}_m(\mathcal{Q}_{g,m})\}$; **if** $F \neq \varnothing$ **then**

      $m \leftarrow \arg\max_{m \in \mathrm{Min}_{\preceq} F} (\texttt{load}_m, \texttt{age}_m)$;

    else if $\mathsf{m} \neq \mathsf{branchRB}$ *and* $a = \max\{j \le |U| : \mathit{SafeAdmit}_g(j)\} > 0$ **then**

      Remove the next a indices i from U; in one device batch create roots s_i with $\ell(s_i) = \varnothing$, reach 1, sampled $X_i \sim \mu$ keyed by $(k, g, i, \mathsf{private})$, and $\iota(s_i) = (g, i, \varnothing)$; $(P, L) \leftarrow \texttt{ChanceClose}_g(\{s_i\}_i, \mathsf{m})$; enqueue P by κ; **continue**;

    else if $\bigsqcup_m \mathcal{Q}_{g,m} \neq \varnothing$ **then**

      $E \leftarrow \{m : \mathcal{Q}_{g,m} \neq \varnothing\}$; $m \leftarrow \arg\max_{m \in \mathrm{Min}_{\preceq} E} (\texttt{fill}_m, \texttt{age}_m)$;

    end

    else

      Set $f_g \leftarrow 1$ and exit the loop: the next fixed root cannot be admitted under the declared transaction; do not replace, retry, or reroll it;

    end

    $B \leftarrow \texttt{Pack}_m(\mathcal{Q}_{g,m})$; mark B EVALUATING; $R \leftarrow \texttt{ReserveChildren}_g(B, \mathsf{m})$ `/* logical child/action addresses are fixed before physical packing */`

    foreach $J \in \mathit{PhysicalPlan}_m(B)$ **do**

      $Y_J \leftarrow \texttt{Forward}_m(\texttt{PartitionInputTable}_{g,m}[d(J)]; \theta_k)$;

      $\texttt{DecisionTable}_{g,m}[d(J)].\texttt{oldV} \leftarrow \begin{cases} Y_J.V[\varphi_m(\texttt{realizedType})], & \mathsf{m} = \mathsf{normal}, \\ Y_J.V[1 : Y_m], & \text{otherwise}; \end{cases}$; $F_J \leftarrow \texttt{Materialize}_g(J, Y_J, R, \mathsf{m})$;

      $(P_J, L_J) \leftarrow \texttt{ChanceClose}_g(F_J, \mathsf{m})$; enqueue P_J by κ; $\texttt{Release}(Y_J,$ sampler scratch for J, physical input/output staging for $J)$;

    end

    Mark B DONE and release each parent's mutable public-state and active reach/profile payload once its retained rows and all children are committed; retain only the occurrence's compact topology fields;

    assert(no DONE occurrence owns a mutable public state or active reach/profile);

    assert($\bigsqcup_m \mathcal{Q}_{g,m} =$ {all live queued player occurrences}); assert(no chance occurrence is pending);

  end

  if $f_g = 0$ **then**

    $\texttt{RolloutStorage}_g \leftarrow \texttt{SealRolloutStorage}_g(\mathsf{m})$; seal all stable (m, d) addresses and branched action spans; on seal transaction failure set $f_g \leftarrow 1$; assert(no occurrence owns a mutable public-state handle); compute partition-local fixed decision masses;

  end

  $\texttt{Release}((\mathcal{Q}_{g,m})_m,$ active reaches/profiles, mutable public states, rollout staging, $\texttt{OccurrenceArena}_g)$;

end

$f \leftarrow \texttt{AllReduceMax}_g\, f_g$ in the common control-collective order; **if** $f = 1$ **then**

  release every current partial/sealed store on every rank; atomically checkpoint the last completed iteration and terminate cleanly

end

`/* No model/queue collective occurs during rollout. The boundary failure agreement prevents a later collective deadlock. No clone of` θ_k `is needed: synchronous optimization cannot start until all fixed targets exist. */`

return $(\mathit{RolloutStorage}_g)_{g\in\mathcal{G}}$;

Algorithm 2: $\texttt{RolloutStorage}_g \leftarrow \texttt{SealRolloutStorage}_g(\mathsf{m})$

Input: completed chunked occurrence, partition, action, and leaf arenas; exact logical row counts and immutable schema; no queued or evaluating occurrence

Output: contiguous immutable topology and column blocks with no spare logical rows; source chunks released only after their last copy

Compute exact destination extents from realized counts. Before allocating any destination, reserve transactionally the simultaneous source-chunk capacity, all destination bytes, and the declared bounded seal scratch under the hard phase ceiling; on failure stop cleanly without changing the sampled object; **if** $\mathsf{m} \neq \mathsf{branchRB}$ **then**

  Count player decisions per root in logical chronological order; exclusive-scan into $\texttt{trajectoryPtr}[G_k + 1]$ and scatter each retained (m, d) using its immutable root id and within-trajectory decision ordinal; write the one aligned leaf row per root;

else

  Copy the already stable parent/child-span columns into `BranchedTreeSpan`; validate that the unique root is row zero, every child span is contiguous, and every player action span has identical order;

end

For each partition and leaf column, allocate exactly the realized dense extent required by its dtype/layout and copy chunk spans in stable row order using one bounded reusable scratch span; do not use concatenation or allocator growth; Synchronize seal copies, validate all pointers/extents and the byte ledger, then release source chunks and seal scratch. Retain destination blocks only; `/* The hard sealing peak includes source capacity plus destination capacity plus scratch. Logical payload formulas alone are not an admission bound. */`

return *the immutable sealed* $\mathit{RolloutStorage}_g$;

Algorithm 3: $R \leftarrow \texttt{ReserveChildren}_g(B, \mathsf{m})$

Input: stable logical batch $B = (s_j)_{j=1}^{b}$ of player or chance parents

Output: reservation R containing immutable child spans, logical RNG addresses, and branched player-action spans

$n_j \leftarrow n(s_j)$ for $j = 1, \ldots, b$; prefix-sum $(n_j)_j$; Reserve exactly $\sum_j n_j$ child occurrences in stable (j, u) order and write $\texttt{childBegin}[s_j]$, $\texttt{degree}[s_j] = n_j$; fail rather than reduce n_j when the hard memory transaction cannot be admitted; **if** $o(\ell(s_j)) \in \mathcal{P}$ *and* $\mathsf{m} = \mathsf{branchRB}$ **then**

  For each j, reserve a contiguous span of n_j rows in $\texttt{ActionTable}_{\kappa(\ell(s_j))}$, set its CSR pointer, and align its uth action row with the uth reserved child;

else if $o(\ell(s_j)) \in \mathcal{P}$ **then**

  Assert $n_j = 1$ and address its implicit edge by the parent's stable decision row;

end

foreach $(j, u) : 1 \le u \le n_j$ **do**

  $\iota(s_{j,u}) \leftarrow (\iota(s_j), u)$; $\log \omega(s_{j,u}) \leftarrow \log \omega(s_j) - \log n_j$ in branchRB;

end

return R;

Algorithm 4: $F_J \leftarrow \texttt{Materialize}_g(J, Y_J, R, \mathsf{m})$

Input: one physical parent subspan J, its current-policy output Y_J, and pre-reserved logical child/action addresses

foreach *reserved* (s_j, u) *with* $s_j \in J$ ***in bounded device batches*** **do**

  $X_{j,u} \leftarrow X_{i(j)}$ in sequential modes; otherwise sample $X_{j,u} \sim \mathbb{P}^{T s_j}(\cdot \mid \ell(s_j))$ with its logical key; Sample $e_{j,u}$ from Y_J at $\varphi_m(X_{j,u,o(\ell(s_j))})$ for a player parent, or from $\chi(\cdot \mid X_{j,u}, \ell(s_j))$ for a chance parent; Set

$$f_{j,u}(x) = \begin{cases} Y_J.\pi[e_{j,u}, \varphi_m(x)], & o(\ell(s_j)) \in \mathcal{P}, \\ \chi(e_{j,u} \mid x, \ell(s_j)), & o(\ell(s_j)) = \mathsf{c}; \end{cases}$$

  Materialize the reserved child with public label $\ell(s_j) e_{j,u}$ and

$$r_{s_{j,u},q} = r_{s_j,q}\ (q \neq o(\ell(s_j))), \qquad r_{s_{j,u},o(\ell(s_j))} = \mathrm{Normalize}\Big(r_{s_j,o(\ell(s_j))} \odot f_{j,u}(-)\Big).$$

  For a player parent, write $e_{j,u}$ and

$$\texttt{oldLogP} \leftarrow \begin{cases} \log Y_J.\pi[e_{j,u}, \varphi_m(\texttt{realizedType})], & \mathsf{m} = \mathsf{normal}, \\ \log Y_J.\pi[e_{j,u}, 1 : Y_m], & \text{otherwise} \end{cases}$$

  to its reserved action row (folded into its decision row in sequential modes);

end

Discard branched posterior profiles $X_{j,u}$ and every unselected old-policy quantity not required by a registered loss; **return** *the reserved children* F_J *in logical order*;

Algorithm 5: $(P, L) \leftarrow \texttt{ChanceClose}_g(F, \mathsf{m})$

Input: new occurrences F

Output: queued player occurrences P and stopped leaf rows L

$P, L \leftarrow \varnothing$; **while** $F \neq \varnothing$ **do**

  Split $F = F_T \sqcup F_P \sqcup F_C$ by $\ell(s) \in \mathcal{T}_{\mathsf{m}}$, player owner, or chance owner; **foreach** $s \in F_P$ **do**

    $m \leftarrow \kappa(\ell(s))$; append stable aligned rows (m, d) to $\texttt{DecisionTable}_m$ and $\texttt{PartitionInputTable}_m$; write owner, normal-mode realized type, $\log \omega(s)$ when branched, and the natural $\texttt{Input}_m(\ell(s), r_s)$; store (m, d) in s; append s to P;

  end

  foreach $s \in F_T$ **do**

    Append one leaf row. In normal mode write the exact sampled terminal reward; in either RB mode write the exact generated reach, immutable completion input, and logical completion key. Store the leaf address in s and append it to L; release its mutable public-state and active reach/profile payload;

  end

  $F \leftarrow \varnothing$; **foreach** *stable maximum-feasible* $C \subseteq F_C$ **do**

    $R_C \leftarrow \texttt{ReserveChildren}_g(C, \mathsf{m})$; $F \leftarrow F \sqcup \texttt{Materialize}_g(C, \perp, R_C, \mathsf{m})$; release every $s \in C$'s mutable public-state and active reach/profile payload after all its children are committed `/* chance needs no model output */`

  end

end

return (P, L);